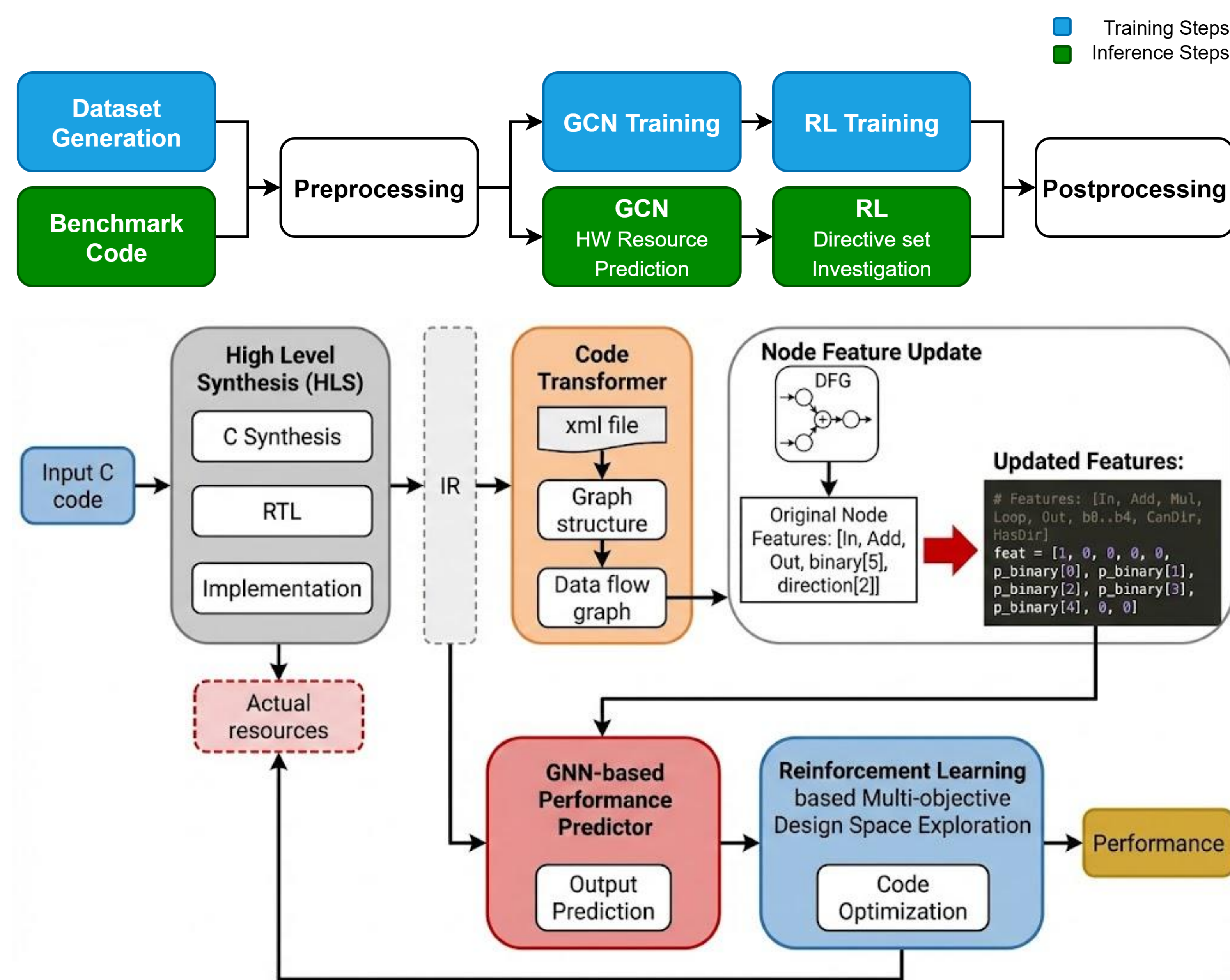


Introduction

High Level Synthesis (HLS)는 설계 생산성을 크게 향상시키지만, 생성된 RTL 설계 결과는 성능 및 자원 효율 측면에서 한계가 있어 지시어(Directive)를 활용한 수동 최적화가 요구된다. 기존의 RTL 자원 사용량 예측 및 지시어 기반 자동 최적화 프레임워크인 IronMan-Pro는 연산자 수준의 최적화에 초점을 두고 있어, 반복문 구조를 포함한 복잡한 연산 패턴에 대한 적용에는 한계가 있었다. 이에 본 연구는 이러한 한계를 극복하기 위해 반복문에 대한 최적화까지 범위를 확장하여, 행렬 곱과 같은 복잡한 연산에서도 효과적인 성능 향상을 달성할 수 있는 범용 HLS 프레임워크를 제안한다.

Method

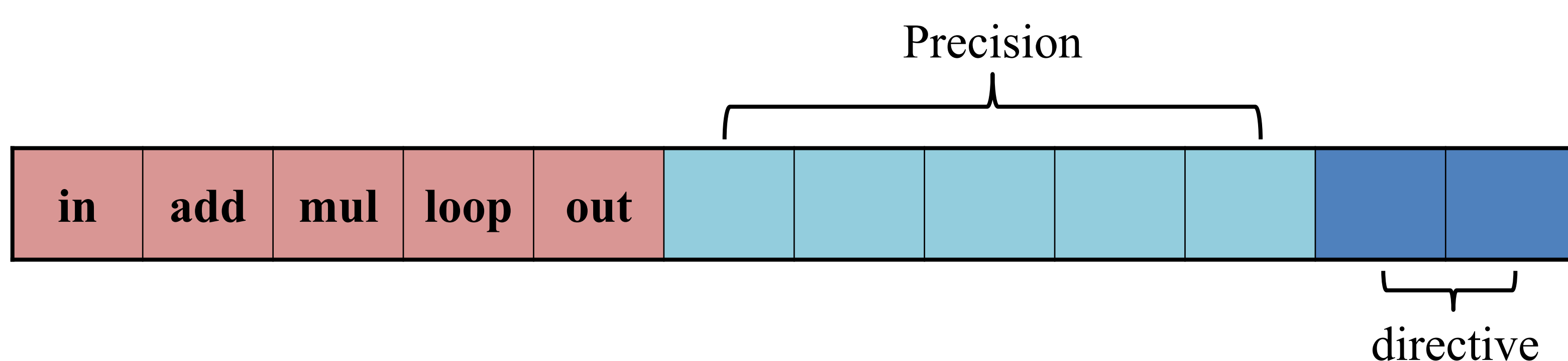
Workflow Overview



본 연구는 기존의 Graph Convolutional Network (GCN) 기반 하드웨어 자원 예측과 Reinforcement Learning (RL) 기반 지시어 탐색 흐름을 유지하며, 모델 학습에 필요한 데이터셋 생성 및 전처리를 중점적으로 확장하였다.

Code 1: Generation of C code containing for-loop

```
while n < max_cnt do
  loop_state == 'NONE'
  Select loop_depth ∈ {1,2,3,4} according to predefined weights
  matrix_mode = (loop_depth == 3) and (random() < 0.7)
  if matrix_mode then
    generate the loops of matrix multiplication with iteration values
    continue
  end if
  Generate the random operation (multiplication or addition)
  Record operation node and edges into DFG
  n += 1
end while
```



입력에 대한 전처리의 경우 총 12bit의 node features를 사용했으며, 반복문 정보가 담긴 loop bit를 추가하였다. 12bit에 맞는 DFG와 edge, node에 대한 정보가 담긴 파일을 생성하며, 이렇게 생성된 정보들이 GCN과 RL의 학습에 사용된다.

Experiment Result

Code 2: Result of generation C code

```
L_n2_1: for(int i=0; i<SIZE; i++) {
  L_n2_2: for(int j=0; j<SIZE; j++) {
    m53 = in_data[j][i] * in_data[4][i];
    m54 = m28 + m31;
    m55 = m26 + in_data[14][j];
    m56 = m36 + m40;
    m20 += (m53 + m54 + m55 + m56);
    m57 = in_data[2][j] * m41;
    m20 += m57;
  }
}
```

Intermediate Operations:
 L1 LOOP outer
 L2 LOOP middle
 in_i_col * INT32 [Parent:L2]
 L3 LOOP inner
 in_k_col * INT32 [Parent:L3]
 m_mult * INT32 [Parent:L3]
 m_sum + INT32 [Parent:L3]

위 그림은 본 연구에서 생성한 학습 데이터의 예시로, C 코드와 이에 대응하는 Data Flow Graph (DFG)를 보여준다. C 코드는 Vitis HLS에서 제안한 perfect loop 구조를 기반으로 작성되었으며, 반복 횟수가 고정된 반복문을 대상으로 한다. 반복문 라벨을 이용해 input, intermediate, output, loop 영역을 구분하였다.

일반적인 산술 연산에는 bind_op 지시어를 적용해 DSP 및 LUT의 사용량을 조정하고, 반복문에는 pipeline 지시어를 적용해 total time을 최소화하는 방향으로 최적화를 수행하였다.

Method	Perfect-loop Performance				
	DSP	LUT	CP (ns)	Latency	Total Time(us)
Vitis HLS	8	2715	7.300	17995	131.363
IronMan-Pro	24	1968	11.432	11689	133.629
Our work	23	2203	7.058	6505	45.912

Method	GEMM Performance				
	DSP	LUT	CP (ns)	Latency	Total Time(ms)
Vitis HLS	14	1931	13.966	2891905	40.39
IronMan-Pro	14	1931	13.966	2891905	40.39
Our work	14	1986	7.297	4202625	30.67

Perfect-loop 실험 결과, 지시어가 없는 환경의 Vitis HLS 합성 결과 대비 실행 시간이 약 65% 단축된 46.9us를 달성하였다. 또한 기존 모델보다 DSP 사용량을 23개로 줄이면서도 동일 수준의 연산 성능을 유지하였다.

GEMM 알고리즘 실험에서는 10ns의 타이밍 제약 조건 하에서 7.297ns의 Critical Path (CP)를 확보하였으며, 이에 따라 동작 클럭이 향상되어 최종 실행 시간이 약 24% 단축되었다. Latency가 증가하였으나, 전체 실행 시간 개선 효과가 더 크게 나타났다. LUT는 약 2.8% 증가하였다.

Conclusion

본 연구에서 제안하는 프레임워크는 최적화 범위를 단순 덧셈·곱셈 연산자 수준에서 반복문 구조까지 확장하여, 기존 IronMan-Pro 프레임워크 대비 우수한 솔루션 탐색 성능을 달성하였다. 이를 위해 반복문 중첩 패턴을 반영한 데이터셋 생성과 Entropy 기반 Reinforcement Learning을 결합하여 설계의 다양성을 극대화하였다. 결과적으로 본 연구는 Matrix Multiplication과 같은 반복 연산 알고리즘에 대해 사용자 수요에 부합하는 면적-성능 균형의 Pareto Solution을 제공하며, 다양한 입력 패턴에 대응 가능한 범용 HLS 튜닝 도구로서의 활용 가능성을 제시한다.